```
C:\Users\Jane Eyre>cd Desktop

C:\Users\Jane Eyre\Desktop>cd Python-Study

C:\Users\Jane Eyre\Desktop\Python-Study>cd
C:\Users\Jane Eyre\Desktop\Python-Study

C:\Users\Jane Eyre\Desktop\Python-Study>python
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> os.getcwd()
'C:\\Users\\Jane Eyre\\Desktop\\Python-Study'
>>> os.listdir()
['.ipynb_checkpoints', 'Exercise Files', 'foo.py', 'hello.py', 'meeting2.ipynb', 'Meeting3.ipynb']
>>> os.listdir()
['.ipynb_checkpoints', 'enable1.txt', 'Exercise Files', 'foo.py', 'hello.py', 'meeting2.ipynb',
'Meeting3.ipynb']
>>> f = open('enable1.txt', 'r')
>>> enable = f.read()
>>> f.close()
>>> type(enable)

>>> len(enable)
1743327
>>> enable[:30]
'aa\naah\naahed\naahing\naahs\naal\na'
>>> print(enable[:30])
aa
aah
aahed
aahing
aahs
aal
a
>>> enable.split()[:20]
['aa', 'aah', 'aahed', 'aahing', 'aahs', 'aal', 'aalii', 'aaliis', 'aals', 'aardvark', 'aardvarks', 'aardwolf',
'aardwolves','aargh', 'aarrgh', 'aarrghh', 'aas', 'aasvogel', 'aasvogels', 'ab']
>>> enable.split()[-20:]
['zymases', 'zyme', 'zymes', 'zymogen', 'zymogene', 'zymogenes', 'zymogens', 'zymogram', 'zymograms',
'zymologies', 'zymology', 'zymosan', 'zymosans', 'zymoses', 'zymosis', 'zymotic', 'zymurgies', 'zymurgy',
'zyzzyva', 'zyzzyvas']
>>> words = enable.split()
>>> words[:10]
['aa', 'aah', 'aahed', 'aahing', 'aahs', 'aal', 'aalii', 'aaliis', 'aals', 'aardvark']
>>> len(words)
172820
>>> len(enable)
1743327
>>> 'syntax' in words
True
>>> 'syntactician' in words
False
>>> sorted(words, key=len, reverse=True)[:10]
['ethylenediaminetetraacetates', 'electroencephalographically', 'ethylenediaminetetraacetate',
'immunoelectrophoretically', 'phosphatidylethanolamines', 'dichlorodifluoromethanes',
'electrocardiographically', 'electroencephalographers', 'electroencephalographies', 'intercomprehensibilities']
>>> [x for x in sorted(words, key=len, reverse=True)[:10]]
['ethylenediaminetetraacetates', 'electroencephalographically', 'ethylenediaminetetraacetate',
'immunoelectrophoretically', 'phosphatidylethanolamines', 'dichlorodifluoromethanes',
'electrocardiographically', 'electroencephalographers', 'electroencephalographies', 'intercomprehensibilities']
>>> [(x, len(s)) for x in sorted(words, key=len, reverse=True)[:10]]
Traceback (most recent call last):
  File "", line 1, in
    [(x, len(s)) for x in sorted(words, key=len, reverse=True)[:10]]
             ^
NameError: name 's' is not defined. Did you mean: 'os'?
>>> [(x, len(x)) for x in sorted(words, key=len, reverse=True)[:10]]
[('ethylenediaminetetraacetates', 28), ('electroencephalographically', 27), ('ethylenediaminetetraacetate', 27),
('immunoelectrophoretically', 25), ('phosphatidylethanolamines', 25), ('dichlorodifluoromethanes', 24),
('electrocardiographically', 24), ('electroencephalographers', 24), ('electroencephalographies', 24),
```

```
('intercomprehensibilities', 24)]
>>> sorted(words, key=lambda x: x.count('e'), reverse=True)[:10]
KeyboardInterrupt
>>> 'beekeeper'.count('e')
5
>>> sorted(words, key=lambda x: x.count('e'), reverse=True)[:10]
['ethylenediaminetetraacetate', 'ethylenediaminetetraacetates', 'defenselessnesses', 'degeneratenesses',
'feeblemindednesses', 'heterogeneousnesses', 'hexamethylenetetramine', 'hexamethylenetetramines',
'levelheadednesses', 'regeneratenesses']
>>> [(x, x.count('e')) for x in sorted(words, key=lambda x: x.count('e'), reverse=True)[:10]]
[('ethylenediaminetetraacetate', 7), ('ethylenediaminetetraacetates', 7), ('defenselessnesses', 6),
('degeneratenesses', 6), ('feeblemindednesses', 6), ('heterogeneousnesses', 6), ('hexamethylenetetramine', 6),
('hexamethylenetetramines', 6), ('levelheadednesses', 6), ('regeneratenesses', 6)]
>>> def isPalin(word):
...     return word == word[::-1]
...
>>> isPalin('hello')
False
>>> isPalin('a')
True
>>> isPalin('kayak')
True
>>> [x for x in words if isPalin(x)][:20]
['aa', 'aba', 'abba', 'aga', 'aha', 'ala', 'alula', 'ama', 'ana', 'anna', 'ava', 'awa', 'bib', 'bob', 'boob',
'bub', 'civic','dad', 'deed', 'deified']
>>> pals = [x for x in words if isPalin(x)]
>>> len(pals)
103
>>> pals
['aa', 'aba', 'abba', 'aga', 'aha', 'ala', 'alula', 'ama', 'ana', 'anna', 'ava', 'awa', 'bib', 'bob', 'boob',
'bub', 'civic','dad', 'deed', 'deified', 'deked', 'deled', 'denned', 'dewed', 'did', 'dud', 'eke', 'eme', 'ere',
'eve', 'ewe', 'eye', 'gag','gig', 'hah', 'halalah', 'hallah', 'heh', 'huh', 'kaiak', 'kayak', 'keek', 'kook',
'level', 'madam', 'marram', 'mem', 'mim', 'minim', 'mm', 'mom', 'mum', 'naan', 'nan', 'noon', 'nun', 'oho',
'otto', 'oxo', 'pap', 'peep', 'pep', 'pip', 'poop', 'pop', 'pullup', 'pup', 'radar', 'redder', 'refer',
'reifier', 'repaper', 'reviver', 'rotator', 'rotor', 'sagas', 'sees', 'selles', 'sememes', 'semes', 'seres',
'sexes', 'shahs', 'sis', 'solos', 'sos', 'stats', 'stets', 'sulus', 'tat', 'tenet', 'terret', 'tet', 'tit',
'toot', 'torot', 'tot', 'tut', 'ulu', 'vav', 'waw', 'wow', 'yay']
>>> [(len(x), x) for x in pals]
[(2, 'aa'), (3, 'aba'), (4, 'abba'), (3, 'aga'), (3, 'aha'), (3, 'ala'), (5, 'alula'), (3, 'ama'), (3, 'ana'),
(4, 'anna'), (3, 'ava'), (3, 'awa'), (3, 'bib'), (3, 'bob'), (4, 'boob'), (3, 'bub'), (5, 'civic'), (3, 'dad'),
(4, 'deed'), (7, 'deified'),(5, 'deked'), (5, 'deled'), (6, 'denned'), (5, 'dewed'), (3, 'did'), (3, 'dud'), (3,
'eke'), (3, 'eme'), (3, 'ere'), (3, 'eve'), (3, 'ewe'), (3, 'eye'), (3, 'gag'), (3, 'gig'), (3, 'hah'), (7,
'halalah'), (6, 'hallah'), (3, 'heh'), (3, 'huh'), (5, 'kaiak'), (5, 'kayak'), (4, 'keek'), (4, 'kook'), (5,
'level'), (5, 'madam'), (6, 'marram'), (3, 'mem'), (3, 'mim'), (5, 'minim'), (2, 'mm'), (3, 'mom'), (3, 'mum'),
(4, 'naan'), (3, 'nan'), (4, 'noon'), (3, 'nun'), (3, 'oho'), (4, 'otto'), (3, 'oxo'), (3, 'pap'), (4, 'peep'),
(3, 'pep'), (3, 'pip'), (4, 'poop'), (3, 'pop'), (6, 'pullup'), (3, 'pup'), (5, 'radar'), (6, 'redder'), (5,
'refer'), (7, 'reifier'), (7, 'repaper'), (7, 'reviver'), (7, 'rotator'), (5, 'rotor'), (5, 'sagas'), (4,
'sees'), (6, 'selles'), (7, 'sememes'), (5, 'semes'), (5, 'seres'), (5, 'sexes'), (5, 'shahs'), (3, 'sis'), (5,
'solos'), (3, 'sos'), (5, 'stats'), (5, 'stets'), (5, 'sulus'), (3, 'tat'), (5, 'tenet'), (6, 'terret'), (3,
'tet'), (3, 'tit'), (4, 'toot'), (5, 'torot'), (3, 'tot'), (3, 'tut'), (3, 'ulu'), (3, 'vav'), (3, 'waw'), (3,
'wow'), (3, 'yay')]
>>> sorted(pals, key=len, reverse=True)
['deified', 'halalah', 'reifier', 'repaper', 'reviver', 'rotator', 'sememes', 'denned', 'hallah', 'marram',
'pullup', 'redder', 'selles', 'terret', 'alula', 'civic', 'deked', 'deled', 'dewed', 'kaiak', 'kayak', 'level',
'madam', 'minim', 'radar', 'refer', 'rotor', 'sagas', 'semes', 'seres', 'sexes', 'shahs', 'solos', 'stats',
'stets', 'sulus', 'tenet', 'torot', 'abba', 'anna', 'boob', 'deed', 'keek', 'kook', 'naan', 'noon', 'otto',
'peep', 'poop', 'sees', 'toot', 'aba', 'aga', 'aha', 'ala', 'ama', 'ana', 'ava', 'awa', 'bib', 'bob', 'bub',
'dad', 'did', 'dud', 'eke', 'eme', 'ere', 'eve', 'ewe', 'eye', 'gag', 'gig', 'hah', 'heh', 'huh', 'mem', 'mim',
'mom', 'mum', 'nan', 'nun', 'oho', 'oxo', 'pap', 'pep', 'pip', 'pop', 'pup', 'sis', 'sos', 'tat', 'tet', 'tit',
'tot', 'tut', 'ulu', 'vav', 'waw', 'wow', 'yay', 'aa', 'mm']
>>> len('deified')
7
>>> [len(x), x for x in sorted(pals, key=len, reverse=True)]
  File "", line 1
    [len(x), x for x in sorted(pals, key=len, reverse=True)]
     ^^^^^^^^^
SyntaxError: did you forget parentheses around the comprehension target?
>>> [(len(x), x) for x in sorted(pals, key=len, reverse=True)]
[(7, 'deified'), (7, 'halalah'), (7, 'reifier'), (7, 'repaper'), (7, 'reviver'), (7, 'rotator'), (7, 'sememes'),
(6, 'denned'), (6, 'hallah'), (6, 'marram'), (6, 'pullup'), (6, 'redder'), (6, 'selles'), (6, 'terret'), (5,
'alula'), (5, 'civic'), (5, 'deked'), (5, 'deled'), (5, 'dewed'), (5, 'kaiak'), (5, 'kayak'), (5, 'level'), (5,
'madam'), (5, 'minim'), (5, 'radar'), (5, 'refer'), (5, 'rotor'), (5, 'sagas'), (5, 'semes'), (5, 'seres'), (5,
```

```
'sexes'), (5, 'shahs'), (5, 'solos'), (5, 'stats'), (5, 'stets'), (5, 'sulus'), (5, 'tenet'), (5, 'torot'), (4,
'abba'), (4, 'anna'), (4, 'boob'), (4, 'deed'), (4, 'keek'), (4, 'kook'), (4, 'naan'), (4, 'noon'), (4, 'otto'),
(4, 'peep'), (4, 'poop'), (4, 'sees'), (4, 'toot'), (3, 'aba'), (3, 'aga'), (3, 'aha'), (3, 'ala'), (3, 'ama'),
(3, 'ana'), (3, 'ava'), (3, 'awa'), (3, 'bib'), (3, 'bob'), (3, 'bub'), (3, 'dad'), (3, 'did'), (3, 'dud'), (3,
'eke'), (3, 'eme'), (3, 'ere'), (3, 'eve'), (3, 'ewe'), (3, 'eye'), (3, 'gag'), (3, 'gig'), (3, 'hah'), (3,
'heh'), (3, 'huh'), (3, 'mem'), (3, 'mim'), (3, 'mom'), (3, 'mum'), (3, 'nan'), (3, 'nun'), (3, 'oho'), (3,
'oxo'), (3, 'pap'), (3, 'pep'), (3, 'pip'), (3, 'pop'), (3, 'pup'), (3, 'sis'), (3, 'sos'), (3, 'tat'), (3,
'tet'), (3, 'tit'), (3, 'tot'), (3, 'tut'), (3, 'ulu'), (3, 'vav'), (3, 'waw'), (3, 'wow'), (3, 'yay'), (2,
'aa'), (2, 'mm')]
>>> novowel = [x for x in words if 'a' not in x and 'e' not in x and 'i' not in x and ]
KeyboardInterrupt
>>> def novowel(word):
...     vcount = sum([1 for x in word if x in 'aeiouy'])
...     return vcount
...
>>> novowel('hello')
2
>>> novowel('lalalalalalay')
7
>>> def novowel(word):
...     vcount = sum([1 for x in word if x in 'aeiouy'])
...     return vcount == 0
...
>>> novowel('lalalalalalay')
False
>>> novowel('shhhhhh')
True
>>> nov_words = [x for x in words if novowel(x)]
>>> nov_words[:10]
['brr', 'brrr', 'crwth', 'crwths', 'cwm', 'cwms', 'hm', 'hmm', 'mm', 'nth']
>>> len(nov_words)
20
>>> nov_words
['brr', 'brrr', 'crwth', 'crwths', 'cwm', 'cwms', 'hm', 'hmm', 'mm', 'nth', 'pfft', 'phpht', 'pht', 'psst',
'sh', 'shh', 'tsk', 'tsks', 'tsktsk', 'tsktsks']
>>> def novowel(word):
...     return 'a' not in word and 'e' not in word and 'i' not in word and 'o' not in word and 'u' not in word
...
>>> nov_words_y_ok = [x for x in words if novowel(x)]
>>> len(nov_words_y_ok)
121
>>> nov_words_y_ok
['brr', 'brrr', 'by', 'byrl', 'byrls', 'bys', 'crwth', 'crwths', 'cry', 'crypt', 'crypts', 'cwm', 'cwms',
'cyst', 'cysts', 'dry', 'dryly', 'drys', 'fly', 'flyby', 'flybys', 'flysch', 'fry', 'ghyll', 'ghylls', 'glycyl',
'glycyls', 'glyph', 'glyphs', 'gym', 'gyms', 'gyp', 'gyps', 'gypsy', 'hm', 'hmm', 'hymn', 'hymns', 'hyp',
'hyps', 'lymph', 'lymphs', 'lynch', 'lynx', 'mm', 'my', 'myrrh', 'myrrhs', 'myth', 'myths', 'mythy', 'nth',
'nymph', 'nymphs', 'pfft', 'phpht', 'pht', 'ply', 'pry', 'psst', 'psych', 'psychs', 'pygmy', 'pyx', 'rhythm',
'rhythms', 'rynd', 'rynds', 'scry', 'sh', 'shh', 'shy', 'shyly', 'sky', 'sly', 'slyly', 'spry', 'spryly', 'spy',
'sty', 'stymy', 'sylph', 'sylphs', 'sylphy', 'syn', 'sync', 'synch', 'synchs', 'syncs', 'synth', 'synths',
'syph', 'syphs', 'syzygy', 'thy', 'thymy', 'try', 'tryst', 'trysts', 'tsk', 'tsks', 'tsktsk', 'tsktsks', 'typp',
'typps', 'typy', 'why', 'whys', 'wry', 'wryly', 'wych', 'wyn', 'wynd', 'wynds', 'wynn', 'wynns', 'wyns',
'xylyl', 'xylyls', 'xyst','xysts']
>>> [x for x in nov_words_y_ok if x not in nov_words]
['by', 'byrl', 'byrls', 'bys', 'cry', 'crypt', 'crypts', 'cyst', 'cysts', 'dry', 'dryly', 'drys', 'fly',
'flyby', 'flybys', 'flysch', 'fry', 'ghyll', 'ghylls', 'glycyl', 'glycyls', 'glyph', 'glyphs', 'gym', 'gyms',
'gyp', 'gyps', 'gypsy', 'hymn', 'hymns', 'hyp', 'hyps', 'lymph', 'lymphs', 'lynch', 'lynx', 'my', 'myrrh',
'myrrhs', 'myth', 'myths', 'mythy', 'nymph', 'nymphs', 'ply', 'pry', 'psych', 'psychs', 'pygmy', 'pyx',
'rhythm', 'rhythms', 'rynd', 'rynds', 'scry', 'shy', 'shyly', 'sky', 'sly', 'slyly', 'spry', 'spryly', 'spy',
'sty', 'stymy', 'sylph', 'sylphs', 'sylphy', 'syn', 'sync', 'synch', 'synchs', 'syncs', 'synth', 'synths',
'syph', 'syphs', 'syzygy', 'thy', 'thymy', 'try', 'tryst', 'trysts', 'typp', 'typps', 'typy', 'why', 'whys',
'wry', 'wryly', 'wych', 'wyn', 'wynd', 'wynds', 'wynn', 'wynns', 'wyns', 'xylyl', 'xylyls', 'xyst', 'xysts']
>>> set(nov_words_y_ok) - set(nov_words)
{'wyns', 'gyms', 'myrrhs', 'gyps', 'tryst', 'glyphs', 'ghyll', 'cyst', 'dry', 'myths', 'psychs', 'hymn',
'sylphy', 'shyly', 'typy', 'trysts', 'flysch', 'glyph', 'sylphs', 'byrls', 'cysts', 'hymns', 'my', 'wych',
'synth', 'rhythms', 'syphs', 'wryly', 'gypsy', 'lynx', 'wry', 'fry', 'rynd', 'sync', 'ghylls', 'synchs',
'lynch', 'drys', 'wynd', 'gym', 'lymph', 'synths', 'dryly', 'hyp', 'thy', 'rynds', 'slyly', 'ply', 'scry',
'wynn', 'hyps', 'cry', 'mythy', 'why', 'myth', 'typps', 'whys', 'pygmy', 'xyst', 'pyx', 'syzygy', 'shy',
'wynns', 'synch', 'myrrh', 'by', 'thymy', 'xysts', 'syncs', 'fly', 'psych', 'glycyl', 'sty', 'xylyls',
'glycyls', 'try', 'sylph', 'wynds', 'crypt', 'nymphs', 'wyn', 'sly', 'sky', 'spry', 'stymy', 'crypts', 'rhythm',
'syn', 'lymphs', 'spryly', 'nymph', 'spy', 'syph', 'gyp', 'flyby', 'typp', 'xylyl', 'flybys', 'byrl', 'pry',
'bys'}
>>> words[:20]
```

```
['aa', 'aah', 'aahed', 'aahing', 'aahs', 'aal', 'aalii', 'aaliis', 'aals', 'aardvark', 'aardvarks', 'aardwolf',
'aardwolves','aargh', 'aarrgh', 'aarrghh', 'aas', 'aasvogel', 'aasvogels', 'ab']
>>> wordle_words = [x for x in enable if len(x)==5]     # oops, wrong object
>>> len(wordle_words)
0
>>> wordle_words = [x for x in words if len(x)==5]
>>> len(wordle_words)
8636
>>> wordle_words[:50]
['aahed', 'aalii', 'aargh', 'abaca', 'abaci', 'aback', 'abaft', 'abaka', 'abamp', 'abase', 'abash', 'abate',
'abbas', 'abbes', 'abbey', 'abbot', 'abeam', 'abele', 'abets', 'abhor', 'abide', 'abler', 'ables', 'abmho',
'abode', 'abohm', 'aboil', 'aboma', 'aboon', 'abort', 'about', 'above', 'abris', 'abuse', 'abuts', 'abuzz',
'abyes', 'abysm', 'abyss', 'acari', 'acerb', 'aceta', 'ached', 'aches', 'achoo', 'acids', 'acidy', 'acing',
'acini', 'ackee']
>>> wordle_words[-50:]
['zamia', 'zanza', 'zappy', 'zarfs', 'zaxes', 'zayin', 'zazen', 'zeals', 'zebec', 'zebra', 'zebus', 'zeins',
'zerks', 'zeros', 'zests', 'zesty', 'zetas', 'zibet', 'zilch', 'zills', 'zincs', 'zincy', 'zineb', 'zings',
'zingy', 'zinky', 'zippy', 'ziram', 'zitis', 'zizit', 'zlote', 'zloty', 'zoeae', 'zoeal', 'zoeas', 'zombi',
'zonal', 'zoned', 'zoner', 'zones', 'zonks', 'zooid', 'zooks', 'zooms', 'zoons', 'zooty', 'zoril', 'zoris',
'zowie', 'zymes']
>>> [x for x in wo>>> [x for x in wordle_words if x[2]=='a' and 't' in x and 'e' in x and 'd' in x and 'p'
 not in x and not in x and 'n' not in x] and 'n' not in x]
  File "", line 1
    [x for x in wordle_words if x[2]=='a' and 't' in x and 'e' in x and 'd' in x and 'p' not in x and 'r' not in
x 'i' not inx and 'n' not in x]

^^^
SyntaxError: invalid syntax
>>> [x for x in wordle_words if x[2]=='a' and 't' in x and 'e' in x and 'd' in x and 'p' not in x and 'r'
 not in x and 'i' not in x and 'n' not in x]]
['dealt', 'death', 'stade', 'tsade']
>>> 'street'
'street'
>>> 'retest'
'retest'
>>> sorted('street')
['e', 'e', 'r', 's', 't', 't']
>>> sorted('retest')
['e', 'e', 'r', 's', 't', 't']
>>> [x for x in words if sorted(x) == sorted('street')]
['retest', 'setter', 'street', 'tester']
>>> 'cried'
'cried'
>>> len('cried')
5
>>> set('cried')
{'e', 'r', 'c', 'd', 'i'}
>>> set('hello')
{'l', 'h', 'e', 'o'}
>>> len('hello')
5
>>> len(set('hello'))
4
>>> nodup = [x for x in words if len(x) == len(set(x))]
>>> len(nodup)
34816
>>> nodup[:20]
['ab', 'abdomen', 'abdomens', 'abduce', 'abducens', 'abducent', 'abduces', 'abducing', 'abduct', 'abducting',
'abduction', 'abductions', 'abductor', 'abductores', 'abductors', 'abducts', 'abed', 'abelmosk', 'abet',
'abets']
>>> nodup[-20:]
['zwieback', 'zwiebacks', 'zydeco', 'zydecos', 'zygoid', 'zygoma', 'zygomas', 'zygomatic', 'zygose', 'zygote',
'zygotes', 'zygotic', 'zymase', 'zyme', 'zymes', 'zymogen', 'zymogens', 'zymosan', 'zymotic', 'zymurgies']
>>> sorted(nodup, key=len, reverse=True)[:10]
['dermatoglyphics', 'uncopyrightable', 'ambidextrously', 'dermatoglyphic', 'troublemakings', 'consumptively',
'copyrightable', 'documentarily', 'endolymphatic', 'flowchartings']
>>> [(len(x), x) for x in sorted(nodup, key=len, reverse=True)[:10]]
[(15, 'dermatoglyphics'), (15, 'uncopyrightable'), (14, 'ambidextrously'), (14, 'dermatoglyphic'), (14,
'troublemakings'), (13, 'consumptively'), (13, 'copyrightable'), (13, 'documentarily'), (13, 'endolymphatic'),
(13, 'flowchartings')]
```

```
>>> import nltk
>>> foo = nltk.corpus.stopwords()
Traceback (most recent call last):
  File "", line 1, in
    foo = nltk.corpus.stopwords()
TypeError: 'LazyCorpusLoader' object is not callable
>>> from nltk.corpus import stopwords
>>> stopwords.words()
['tyre', 'rreth', 'le', 'atyre', 'këta', 'megjithëse', 'kemi', 'per', 'ndonëse', 'dytë', 'pse', 'tha', 'aty',
'ndaj', 'ke', 'këtë', 'duhet', 'pa', 'perket', 'veç', 'ndonje', 'një', 'keshtu', 's', 'janë', 'jane', 'ti',
'ia', 'megjithese', 'prej', 'ishte', 'tjerë', 'ai', 'se', 'tillë', 'do', 'si', 'ja', 'tonë', 'keta', 'pastaj',
'ndersa', 'siç', 'unë', 'gjate', 'di', 'kësaj', 'cilin', 'kjo', 'dhënë', 'da', 'teper', 'ketij', 'ama', 'pasi',
'fjalë', 'kanë', 'vetem', 'za', 'd.m.th.', 'ose', 'pas', 'ndonjë', 'cila', 'ndodhur', 'dyte', 'ardhur', 'kësi',
'nga', 'vete', 'atij', 'ta', 'jenë', 'rendit', 'tane', 'keso', 'deri', 'tone', 'të', 'prandaj', 'bëjë',
'domethënë', 'dhe', 'qi', 'mirepo', 'tona', 'që', 'u', 'këtu', 'cilet', 'jene', 'tjere', 'gjë', 'së', 'gjatë',
'duhej', 't', 'dhene', 'thuhet', 'po', 'une', 'dy', 'cfare', 'ndërsa', 'sepse', 'edhe', 'cilen', 'to',
'meqenese', 'meje', 'tij', 'qene', 'jeni', 'them', 'përket', 'keto', 'ni', 'këso', 'asaj', 'ajo', 'sic',
'vetëm', 'ketyre', 'andaj', 'na', 'sa', 'kesaj', 'cili', 'këtyre', 'domethene', 'mirëpo', 'cilën', 'mos',
'madh', 'qenë', 'cilët', 'thënë', 'jemi', 'fjale', 'soje', 'neve', 'gjitha', 'kështu', 'vet', 'kur', 'ty',
'meqë', 'meqenëse', 'jush', 'ketë', 'para', 'צמהן', 'מי', 'מה', 'איפה', 'היכן', 'במקום שבו', 'אם', 'לאן', 'למקום שבו',
'מקום בו', 'איזה', 'מהיכן', 'איך', 'כיצד', 'באיזו מידה', 'מתי', 'בשעה ש', 'כאשר', 'כש', 'למרות', 'לפני', 'אחרי', 'מאיזו סיבה',
'הסיבה שבגללה', 'למה', 'מדוע', 'לאיזו תכלית', 'כי', 'יש', 'אין', 'אך', 'מנין', 'מאין', 'מאיפה', 'יכל', 'יכלה', 'יכלו', 'יכול',
'יכולה', 'יכולים', 'יכולות', 'יוכלו', 'יוכל', 'מסוגל', 'לא', 'רק', 'אולי', 'אין', 'לאו', 'אי', 'כלל', 'נגד', 'אם', 'עם', 'אל',
'אלה', 'אלו', 'אף', 'על', 'מעל', 'מתחת', 'מצד', 'בשביל', 'לבין', 'באמצע', 'בתוך', 'דרך', 'מבעד', 'באמצעות', 'למעלה', 'למטה',
'מחוץ', 'מן', 'לעבר', 'מכאן', 'כאן', 'הנה', 'הרי', 'פה', 'שם', 'אך', 'ברם', 'שוב', 'אבל', 'מבלי', 'בלי', 'מלבד', 'רק',
'בגלל', 'מכיוון', 'עד', 'אשר', 'ואילו', 'למרות', 'אס', 'כמו', 'כפי', 'אז', 'אחרי', 'כן', 'לכן', 'לפיכך', 'מאד', 'עז', 'מעט',
'או', 'אחרות', 'אשר', 'אחרים', 'מעטים'a', 'במידה', 'שוב', 'יותר', 'מדי', 'גם', 'כן', 'נו', 'אחר', 'אחרת', 'אחרים', 'aadi', 'aaj',
'aap', 'aapne', 'aata', 'aati', 'aaya', 'aaye', 'ab', 'abbe', 'abbey', 'abe', 'abhi', 'able', 'about', 'above',
'accha', 'according', 'accordingly', 'acha', 'achcha', 'across', 'actually', 'after', 'afterwards', 'again',
'against', 'agar', 'ain', 'aint', "ain't", 'aisa', 'aise', 'aisi', 'alag', 'all', 'allow', 'allows', 'almost',
'alone', 'along', 'already', 'also', 'although', 'always', 'am', 'among', 'amongst', 'an', 'and', 'andar',
'another', 'any', 'anybody', 'anyhow', 'anyone', 'anything', 'anyway', 'anyways', 'anywhere', 'ap', 'apan',
'apart', 'apna', 'apnaa', 'apne', 'apni', 'appear', 'are', 'aren', 'arent', "aren't", 'around', 'arre', 'as',
'aside', 'ask', 'asking', 'at', 'aur', 'avum', 'aya', 'aye', 'baad', 'baar', 'bad', 'bahut', 'bana', 'banae',
'banai', 'banao', 'banaya', 'banaye', 'banayi', 'banda', 'bande', 'bandi', 'bane', 'bani', 'bas', 'bata',
'batao', 'bc', 'be', 'became', 'because', 'become', 'becomes', 'becoming', 'been', 'before', 'beforehand',
'behind', 'being', 'below', 'beside', 'besides', 'best', 'better', 'between', 'beyond', 'bhai', 'bheetar',
'bhi', 'bhitar', 'bht', 'bilkul', 'bohot', 'bol', 'bola', 'bole', 'boli', 'bolo', 'bolta', 'bolte', 'bolti',
'both', 'brief', 'bro', 'btw', 'but', 'by', 'came', 'can', 'cannot', 'cant', "can't", 'cause', 'causes',
'certain', 'certainly', 'chahiye', 'chaiye', 'chal', 'chalega', 'chhaiye', 'clearly', "c'mon", 'com', 'come',
'comes', 'could', 'couldn', 'couldnt', "couldn't", 'd', 'de', 'dede', 'dega', 'degi', 'dekh', 'dekha', 'dekhe',
'dekhi', 'dekho', 'denge', 'dhang', 'di', 'did', 'didn', 'didnt', "didn't", 'dijiye', 'diya', 'diyaa', 'diye',
'diyo', 'do', 'does', 'doesn', 'doesnt',"doesn't", 'doing', 'done', 'dono', 'dont', "don't", 'doosra', 'doosre',
'down', 'downwards', 'dude', 'dunga', 'dungi', 'during', 'dusra', 'dusre', 'dusri', 'dvaara', 'dvara', 'dwaara',
'dwara', 'each', 'edu', 'eg', 'eight', 'either', 'ek', 'else', 'elsewhere', 'enough', 'etc', 'even', 'ever',
'every', 'everybody', 'everyone', 'everything', 'everywhere', 'ex', 'exactly', 'example', 'except', 'far',
'few', 'fifth', 'fir', 'first', 'five', 'followed', 'following', 'follows', 'for', 'forth', 'four', 'from',
'further', 'furthermore', 'gaya', 'gaye', 'gayi', 'get', 'gets', 'getting', 'ghar', 'given', 'gives', 'go',
'goes', 'going', 'gone', 'good', 'got', 'gotten', 'greetings', 'haan', 'had', 'hadd', 'hadn', 'hadnt', "hadn't",
'hai', 'hain', 'hamara','hamare', 'hamari', 'hamne', 'han', 'happens', 'har', 'hardly', 'has', 'hasn', 'hasnt',
"hasn't", 'have', 'haven', 'havent', "haven't", 'having', 'he', 'hello', 'help', 'hence', 'her', 'here',
'hereafter', 'hereby', 'herein', "here's", 'hereupon', 'hers', 'herself', "he's", 'hi', 'him', 'himself', 'his',
'hither', 'hm', 'hmm', 'ho', 'hoga', 'hoge', 'hogi', 'hona', 'honaa', 'hone', 'honge', 'hongi', 'honi',
'hopefully', 'hota', 'hotaa', 'hote', 'hoti', 'how', 'howbeit', 'however', 'hoyenge', 'hoyengi', 'hu', 'hua',
'hue', 'huh', 'hui', 'hum', 'humein', 'humne', 'hun', 'huye', 'huyi', 'i', "i'd", 'idk', 'ie', 'if', "i'll",
"i'm", 'imo', 'in', 'inasmuch', 'inc', 'inhe', 'inhi', 'inho', 'inka', 'inkaa', 'inke', 'inki', 'inn', 'inner',
'inse', 'insofar', 'into', 'inward', 'is', 'ise', 'isi', 'iska', 'iskaa', 'iske', 'iski', 'isme', 'isn', 'isne',
'isnt', "isn't", 'iss', 'isse', 'issi', 'isski', 'it', "it'd", "it'll", 'itna', 'itne', 'itni', 'itno', 'its',
"it's", 'itself', 'ityaadi', 'ityadi', "i've", 'ja', 'jaa', 'jab', 'jabh', 'jaha', 'jahaan', 'jahan', 'jaisa',
'jaise', 'jaisi', 'jata', 'jayega', 'jidhar', 'jin', 'jinhe', 'jinhi', 'jinho', 'jinhone', 'jinka', 'jinke',
'jinki', 'jinn', 'jis', 'jise', 'jiska', 'jiske', 'jiski',
# ... results clipped

>>> stopwords.words('english')
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren',
"aren't", 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by',
'can', 'couldn', "couldn't", 'd', 'did', 'didn', "didn't", 'do', 'does', 'doesn', "doesn't", 'doing', 'don',
"don't", 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', "hadn't", 'has', 'hasn',
"hasn't", 'have', 'haven', "haven't", 'having', 'he', "he'd", "he'll", 'her', 'here', 'hers', 'herself', "he's",
'him', 'himself', 'his', 'how', 'i', "i'd", 'if', "i'll", "i'm", 'in', 'into', 'is','isn', "isn't", 'it',
```

```
"it'd", "it'll", "it's", 'its', 'itself', "i've", 'just', 'll', 'm', 'ma', 'me', 'mightn', "mightn't", 'more',
'most', 'mustn', "mustn't", 'my', 'myself', 'needn', "needn't", 'no', 'nor', 'not', 'now', 'o', 'of', 'off',
'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same',
'shan', "shan't", 'she', "she'd", "she'll", "she's", 'should', 'shouldn', "shouldn't", "should've", 'so',
'some', 'such', 't', 'than', 'that', "that'll", 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there',
'these', 'they', "they'd", "they'll", "they're", "they've", 'this', 'those', 'through', 'to', 'too', 'under',
'until', 'up', 've', 'very', 'was', 'wasn', "wasn't", 'we', "we'd", "we'll", "we're", 'were', 'weren',
"weren't", "we've", 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won',
"won't", 'wouldn', "wouldn't", 'y', 'you', "you'd", "you'll", 'your', "you're", 'yours', 'yourself',
'yourselves', "you've"]
>>> stopwords.words('spanish')
['de', 'la', 'que', 'el', 'en', 'y', 'a', 'los', 'del', 'se', 'las', 'por', 'un', 'para', 'con', 'no', 'una',
'su', 'al', 'lo', 'como', 'más', 'pero', 'sus', 'le', 'ya', 'o', 'este', 'sí', 'porque', 'esta', 'entre',
'cuando', 'muy', 'sin', 'sobre', 'también', 'me', 'hasta', 'hay', 'donde', 'quien', 'desde', 'todo', 'nos',
'durante', 'todos', 'uno', 'les', 'ni', 'contra', 'otros', 'ese', 'eso', 'ante', 'ellos', 'e', 'esto', 'mí',
'antes', 'algunos', 'qué', 'unos', 'yo', 'otro', 'otras', 'otra', 'él', 'tanto', 'esa', 'estos', 'mucho',
'quienes', 'nada', 'muchos', 'cual', 'poco', 'ella', 'estar', 'estas', 'algunas', 'algo', 'nosotros', 'mi',
'mis', 'tú', 'te', 'ti', 'tu', 'tus', 'ellas', 'nosotras', 'vosotros', 'vosotras', 'os', 'mío', 'mía', 'míos',
'mías', 'tuyo', 'tuya', 'tuyos', 'tuyas', 'suyo', 'suya', 'suyos', 'suyas', 'nuestro', 'nuestra', 'nuestros',
'nuestras', 'vuestro', 'vuestra', 'vuestros', 'vuestras', 'esos', 'esas', 'estoy', 'estás', 'está', 'estamos',
'estáis', 'están', 'esté', 'estés', 'estemos', 'estéis', 'estén', 'estaré', 'estarás', 'estará', 'estaremos',
'estaréis', 'estarán', 'estaría', 'estarías', 'estaríamos', 'estaríais', 'estarían', 'estaba', 'estabas',
'estábamos', 'estabais', 'estaban', 'estuve', 'estuviste', 'estuvo', 'estuvimos', 'estuvisteis', 'estuvieron',
'estuviera', 'estuvieras', 'estuviéramos', 'estuvierais', 'estuvieran', 'estuviese', 'estuvieses',
'estuviésemos', 'estuvieseis', 'estuviesen', 'estando', 'estado', 'estada', 'estados', 'estadas', 'estad','he',
'has', 'ha', 'hemos', 'habéis', 'han', 'haya', 'hayas', 'hayamos', 'hayáis', 'hayan', 'habré', 'habrás',
'habrá', 'habremos', 'habréis', 'habrán', 'habría', 'habrías', 'habríamos', 'habríais', 'habrían', 'había',
'habías', 'habíamos', 'habíais','habían', 'hube', 'hubiste', 'hubo', 'hubimos', 'hubisteis', 'hubieron',
'hubiera', 'hubieras', 'hubiéramos', 'hubierais', 'hubieran', 'hubiese', 'hubieses', 'hubiésemos', 'hubieseis',
'hubiesen', 'habiendo', 'habido', 'habida', 'habidos', 'habidas', 'soy', 'eres', 'es', 'somos', 'sois', 'son',
'sea', 'seas', 'seamos', 'seáis', 'sean', 'seré', 'serás', 'será', 'seremos', 'seréis', 'serán', 'sería',
'serías', 'seríamos', 'seríais', 'serían', 'era', 'eras', 'éramos', 'erais', 'eran', 'fui', 'fuiste','fue',
'fuimos', 'fuisteis', 'fueron', 'fuera', 'fueras', 'fuéramos', 'fuerais', 'fueran', 'fuese', 'fueses',
'fuésemos', 'fueseis', 'fuesen', 'sintiendo', 'sentido', 'sentida', 'sentidos', 'sentidas', 'siente', 'sentid',
'tengo', 'tienes', 'tiene', 'tenemos', 'tenéis', 'tienen', 'tenga', 'tengas', 'tengamos', 'tengáis', 'tengan',
'tendré', 'tendrás', 'tendrá', 'tendremos', 'tendréis', 'tendrán', 'tendría', 'tendrías', 'tendríamos',
'tendríais', 'tendrían', 'tenía', 'tenías', 'teníamos', 'teníais','tenían', 'tuve', 'tuviste', 'tuvo',
'tuvimos', 'tuvisteis', 'tuvieron', 'tuviera', 'tuvieras', 'tuviéramos', 'tuvierais', 'tuvieran', 'tuviese',
'tuvieses', 'tuviésemos', 'tuvieseis', 'tuviesen', 'teniendo', 'tenido', 'tenida', 'tenidos', 'tenidas',
'tened']
>>> 'she' in stopwords.words('english')
True
>>> 'must' in stopwords.words('english')
False
>>> 'would' in stopwords.words('english')
False
>>> 'will' in stopwords.words('english')
True

>>> any([True, False, False])
True
>>> all([True, False, False])
False
>>> any([])
False
>>> any(['hello'])
True
>>> [c in 'aeiouy' for c in 'hello']
[False, True, False, False, True]
>>> any([c in 'aeiouy' for c in 'hello'])
True
>>> [c in 'aeiouy' for c in 'tsktsk']
[False, False, False, False, False, False]
>>> any([c in 'aeiouy' for c in 'tsktsk'])
False
>>> not any([c in 'aeiouy' for c in 'tsktsk'])
True
>>> exit()

C:\Users\Jane Eyre\Desktop\Python-Study>python
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> import os
>>> os.getcwd()
'C:\\Users\\Jane Eyre\\Desktop\\Python-Study'
>>> os.listdir()
['.ipynb_checkpoints', 'enable1.txt', 'Exercise Files', 'foo.py', 'hello.py', 'meeting2.ipynb',
'Meeting3.ipynb']
>>> f = open(r'C:\Users\Jane Eyere\Desktop\tale.txt')
Traceback (most recent call last):
  File "", line 1, in
    f = open(r'C:\Users\Jane Eyere\Desktop\tale.txt')
FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Users\\Jane Eyere\\Desktop\\tale.txt'
>>> f = open(r'C:\Users\Jane Eyre\Desktop\tale.txt')
>>> taletxt = f.read()
>>> f.close()
>>> taletxt[:50]
'It was the best of times, it was the worst of time'
>>> print(taletxt)
It was the best of times, it was the worst of times,
it was the age of wisdom, it was the age of foolishness,
it was the epoch of belief, it was the epoch of incredulity,
it was the season of Light, it was the season of Darkness,
it was the spring of hope, it was the winter of despair,
we had everything before us, we had nothing before us,
we were all going direct to heaven, we were all going direct
the other way - in short, the period was so far like the present
period, that some of its noisiest authorities insisted on its
being received, for good or for evil, in the superlative degree
of comparison only.

>>> f = open(r'../tale.txt')
>>> taletxt = f.read()
>>> f.close()
>>> print(taletxt)
It was the best of times, it was the worst of times,
it was the age of wisdom, it was the age of foolishness,
it was the epoch of belief, it was the epoch of incredulity,
it was the season of Light, it was the season of Darkness,
it was the spring of hope, it was the winter of despair,
we had everything before us, we had nothing before us,
we were all going direct to heaven, we were all going direct
the other way - in short, the period was so far like the present
period, that some of its noisiest authorities insisted on its
being received, for good or for evil, in the superlative degree
of comparison only.

>>> f = open(r'C:\Users\Jane Eyre\Desktop\Python-Study\mycorpus\how-do-i.txt')
>>> howtxt = f.read()
>>> f.close()
>>> howtxt[:200]
'How do I love thee? Let me count the ways.\nI love thee to the depth and breadth and height\nMy soul can reach,
when feelingout of sight\nFor the ends of Being and ideal Grace.\nI love thee to the level '
>>> f = open(r'mycorpus\how-do-i.txt')
>>> howtxt = f.read()
>>> f.close()
>>> howtxt[-200:]
"ldhood's faith.\nI love thee with a love I seemed to lose\nWith my lost saints, I love thee with the
breath,\nSmiles, tears,of all my life! and, if God choose,\nI shall but love thee better after death.\n"
>>>
```